

Question 1: Stack Implementation Using Array

```
class ArrayStack:
    def __init__(self, capacity):
        self.stack = [None] * capacity
        self.top_index = -1 // Represents an empty stack

    def Push(key):
        if top_index == capacity - 1:
            print("Stack Overflow")
        else:
            top_index += 1
            stack[top_index] = key

    def Pop():
        if top_index == -1:
            print("Stack Underflow")
            return None
        else:
            key = stack[top_index]
            top_index -= 1
            return key

    def Top():
        if top_index == -1:
            print("Stack is Empty")
            return None
        else:
            return stack[top_index]
```

Question 1: Stack Implementation Using Array

Questions:

1. Using the given pseudocode, analyze the following:

- What will happen when you attempt to Pop() an element from an empty stack?
- If the array has a capacity of 5 and you attempt to push 6 elements, what would happen? Justify your answer based on the pseudocode.

2. Determine the time complexity of the following operations:

- Push operation
- Pop operation
- Top operation

3. Explain the limitations of using an array for stack implementation, especially in terms of memory allocation.

Question 2: Stack Implementation Using Queue

```
class Node:
    def __init__(self, key):
        self.key = key
        self.next = None

class LinkedListStack:
    def __init__(self):
        self.top = None // Empty stack is represented by a None top

    def Push(key):
        new_node = Node(key)
        new_node.next = self.top
        self.top = new_node

    def Pop():
        if self.top is None:
            print("Stack Underflow")
            return None
        else:
            key = self.top.key
            self.top = self.top.next
            return key

    def Top():
        if self.top is None:
            print("Stack is Empty")
            return None
        else:
            return self.top.key
```

Question 2: Stack Implementation Using Queue

Questions:

1. Using the given pseudocode, analyze the following:

- What happens when the stack is empty and you attempt to Pop an element?
- What is the space complexity of this stack implementation for n elements, considering both data and pointers?

2. Determine the time complexity of the following operations:

- Push operation
- Pop operation
- Top operation

3. Compare this linked list-based implementation with the array-based implementation in terms of:

- Memory usage
- Flexibility (in terms of stack size)
- Performance under different load conditions.

Question 3: Comparative Analysis

Scenario: Suppose you have a system that must handle a large and dynamic number of elements, and the system cannot predict the maximum number of elements in advance.

Question:

Which stack implementation (Array or Linked List) would you choose for this system? Justify your answer based on memory usage, time complexity, and the need for dynamic sizing.